

SuVa Protocol — Technical Brief for Mohammed

March 2026 | William Schulz

Timeline — This Must Be Done in Under 2 Months

This is not a research project with an open-ended timeline. We are building a working protocol with a hard deadline. Everything from Phase 1 through Phase 3 must be complete within 8 weeks from today.

Phase	Deadline
Phase 1 — Tests passing, security report	End of Week 1
Phase 2 — Falcon integrated, tests passing	End of Week 3
Phase 3 — Vault contract on Sepolia, end-to-end working	End of Week 6
Phase 4 — ERC-4337 wallet	End of Week 8

This is achievable. With Claude Code assisting on boilerplate, test generation, and debugging, a focused developer can move through each phase in days not weeks. The codebase is not being built from scratch — ZKNox has done the hard cryptographic lifting. Your job is integration, architecture, and validation.

If you are stuck on something for more than a day, message William immediately. Do not sit on a blocker.

What We Are Building

SuVa is a quantum-proof stablecoin protocol on Ethereum. The goal is to wrap existing stablecoins (USDT, USDC) so that they cannot be stolen or broken by quantum computers.

The cryptographic strategy uses two NIST-standardised post-quantum signature schemes:

- **Falcon (FIPS 206)** — Primary scheme. Lattice-based, ~350K gas on EVM. Used for daily transactions.
- **SPHINCS+ (FIPS 205)** — Fallback scheme. Hash-based, maximum security. Used for high-value protection.

The current codebase is a fork of ZKNox's ETHDILITHIUM. It implements Dilithium (ML-DSA / FIPS 204) on-chain verification in Solidity and off-chain signing in Python. This is the research foundation, not the final product. ZKNox (Renaud Dubois, Simon Masson) have also already built ETHFALCON — we do not need to build Falcon from scratch, we integrate their work.

Honest State of the Codebase

The zip file (falcon_dilithium) contains:

- The ZKNox ETHDILITHIUM-main repo (Solidity + Python)
- A Python virtual environment
- No SuVa-specific code yet

Everything beyond the forked repo needs to be built. That is your work.

The Roadmap — What Needs to Be Built

Phase 1 — Verify the Foundation

Before building anything on top of it, the existing codebase needs to be confirmed working.

What to do:

- Run `forge test` inside ETHDILITHIUM-main. All tests must pass.
- Run `generate_dilithium_test_vectors.py` and `generate_ethdilithium_test_vectors.py`. Confirm both produce valid outputs.
- Cross-validate: feed the Python-generated test vectors into the Solidity forge tests. The on-chain verifier must accept signatures produced by the off-chain Python code.

Questions to work through as you do this:

- Does ETHDilithium's substitution of SHAKE-256 with Keccak-256 affect security? Under what conditions?

- Is the Yul assembly in `ZKNOX_NTT.sol` correct? Are there any edge cases in the modular arithmetic?
- What are the actual measured gas costs on a local Anvil node versus the reported estimates?

Deliverable: A short written report covering what works, any bugs found, and your assessment of the security implications of the Keccak substitution in ETHDilithium.

Phase 2 — Integrate Falcon

ZKNox's ETHFALCON already exists at <https://github.com/ZKNoxHQ/ETHFALCON>. The task is integration, not building from scratch.

What to do:

- Study the ETHFALCON codebase and understand how it differs structurally from ETHDILITHIUM.
- Add ETHFALCON into the SuVa repo (submodule or direct integration).
- Write Python code to generate Falcon keypairs and signatures.
- Write Solidity forge tests that verify Falcon signatures produced by the Python code are accepted by the on-chain verifier.
- Measure actual gas cost on a testnet.

Questions to work through:

- Falcon-512 or Falcon-1024 — which parameter set is appropriate for SuVa's use case and why?
- What is the signature size difference and how does that affect on-chain calldata costs?
- Are there any known implementation vulnerabilities in ETHFALCON that need to be addressed before we build on it?

Deliverable: Falcon verifier integrated, forge tests passing, gas benchmark confirmed, written recommendation on parameter set choice.

Phase 3 — Build the SuVa Vault Contract

This is the core of the protocol. A smart contract that holds USDT or USDC and requires a Falcon signature to authorise any withdrawal or transfer.

What to do:

- Design and implement `QuantumVault.sol` — a contract that accepts ERC-20 deposits and uses the Falcon verifier for authorisation.
- Implement `qUSDT` — an ERC-20 token minted when USDT is deposited, burned on withdrawal.
- Implement on-chain public key registration — users register their Falcon public key linked to their Ethereum address.
- Write an end-to-end integration test: Python generates Falcon keypair, registers on-chain, deposits USDT, signs a withdrawal, verifier accepts, USDT returned.

Questions to work through:

- Should Falcon public keys be stored fully on-chain, or hashed and stored with off-chain availability? What are the gas and security trade-offs?
- How should key rotation work if a user needs to update their Falcon keypair?
- What happens if a user loses their Falcon private key? Is there a recovery mechanism or is the asset permanently locked? This is a design decision that needs to be made explicitly.
- Should the vault be upgradeable (proxy pattern) or immutable? What does each choice mean for security and trust?

Deliverable: Vault contract, qUSDT token, key registration, full end-to-end forge test passing on Sepolia testnet.

Phase 4 — ERC-4337 Account Abstraction Wallet

The SuVa website describes a quantum-secure smart contract wallet that uses both a standard EVM key and a Falcon key together. This is built on ERC-4337.

What to do:

- Build a `SuVaWallet.sol` smart contract wallet implementing the ERC-4337 `IAccount` interface.
- The `validateUserOp` function must verify both the ECDSA signature (standard Ethereum) and the Falcon signature before executing.
- Build a bundler-compatible transaction flow.

Questions to work through:

- Is requiring dual signatures (ECDSA + Falcon) for every transaction the right UX tradeoff, or should Falcon be optional or threshold-based?

- How does the paymaster interact with a quantum-secured wallet? Can gas be paid in USDC from within the vault?
- ERC-4337 userOps have a calldata size limit — does a Falcon signature (~690 bytes) fit comfortably within standard bundler limits?

This phase should not start until Phase 3 is complete.

Phase 5 — SPHINCS+ Fallback (Research)

SPHINCS+ is extremely expensive on EVM mainnet (50M to 500M gas). The focus here is on L2 deployment where gas is 10 to 100x cheaper.

Questions to work through:

- Is there an existing SPHINCS+ EVM implementation we can build on, or does this need to be written from scratch?
- What is the realistic gas cost of SPHINCS+ on Arbitrum and Base at current L2 gas prices?
- Could a ZK proof be used to compress SPHINCS+ verification on-chain (verifying a ZK proof that the SPHINCS+ signature is valid rather than verifying the full signature directly)?

This is a research phase. The output is a feasibility report and proof of concept, not a production contract.

What We Need for Academic Peer Review (UCL Grant)

We are partnering with Prof. Sadrzadeh at UCL as the academic partner on an EPSRC grant. SuVa is the industrial partner. For the work to be grant-eligible and peer-reviewable, the following are required:

- The code must demonstrably follow NIST FIPS 204, 205, and 206. Every deviation must be documented with security analysis.
- The implementation must pass NIST Known Answer Test (KAT) vectors for each scheme.
- A written security analysis justifying the chosen parameter sets.
- Empirical gas measurements on public testnet, not theoretical estimates.
- A clear threat model document: what does SuVa protect against, and what does it not protect against.

The professor validates and guides. Your technical reports need to be rigorous enough for her to review and co-sign.

What Done Looks Like — Summary

Phase	Done when
1	forge test passes, cross-validation passes, security report delivered
2	Falcon integrated, forge tests pass, gas confirmed, parameter recommendation written
3	Vault contract deployed on Sepolia, end-to-end test passes, qUSDT mints and burns
4	SuVa smart wallet deployed, dual-sig validation working, bundler compatible
5	Feasibility report on SPHINCS+ L2 delivery, PoC if viable

Reference Material

Resource	Link
ZKNox ETHDILITHIUM	https://github.com/ZKNoxHQ/ETHDILITHIUM
ZKNox ETHFALCON	https://github.com/ZKNoxHQ/ETHFALCON
NIST FIPS 204 (Dilithium)	https://doi.org/10.6028/NIST.FIPS.204
NIST FIPS 205 (SPHINCS+)	https://doi.org/10.6028/NIST.FIPS.205
NIST FIPS 206 (Falcon)	https://doi.org/10.6028/NIST.FIPS.206
ERC-4337 Account Abstraction	https://eips.ethereum.org/EIPS/eip-4337

Resource	Link
Foundry docs	https://book.getfoundry.sh/

Confidential — SuVa Protocol internal document.